

OFFICIAL PROJECT SUBMISSION REPORT

DISTRIBUTED EVENT-DRIVEN E-COMMERCE MICROSERVICES

Enterprise Microservices Infrastructure & Saga Pattern Specification

Submitted To: ZAALIMA DEVELOPMENT PVT LTD

Date: September 30, 2026

Author: KANNEKANTI TEJA KIRAN

GitHub: github.com/tejaa27/E-Commerce-microservices

1. Executive Summary & Architecture Overview

This project presents a highly scalable, resilient, event-driven back-end infrastructure built for modern E-Commerce platform demands. The system breaks down monolithic workflows into 6 independently deployable microservices (API Gateway, Eureka Registry, Order Processing, Inventory Stock Management, Payment Processing, and Notification Alerts). Data consistency across PostgreSQL and MongoDB datastores is maintained using the Choreography-based Saga Design Pattern over Apache Kafka without distributed 2PC locks.



Figure 1: Event-Driven Choreography Saga Microservices Architecture Diagram

2. Technical Stack Specifications

Layer	Technology / Framework	Role / Functionality
Core Language	Java 17 / 21	Type-safe microservices application logic
Framework	Spring Boot 3.2.0, Spring Cloud	Microservices orchestration & WebFlux Gateway
Service Discovery	Spring Cloud Netflix Eureka	Dynamic service registration & discovery (:8761)
API Gateway	Spring Cloud Gateway	Central entry point & JWT authentication (:8085)
Message Broker	Apache Kafka 7.5.0, Zookeeper	Event streaming & Saga pub/sub messaging

Databases	PostgreSQL 15 & MongoDB 6.0	PostgreSQL: order_db, inventory_db, payment_db; Mongo: notifica
Resilience	Resilience4j	Circuit breakers, rate limiters, fallback handlers
Observability	Micrometer & Zipkin	Distributed tracing span collector (:9411)
Containerization	Docker, Docker Compose, K8s	Multi-stage Dockerfiles & Kubernetes manifests

3. System Dashboards & Tracing Verification



Figure 2: Spring Cloud Eureka Service Registry Dashboard (<http://localhost:8761>)

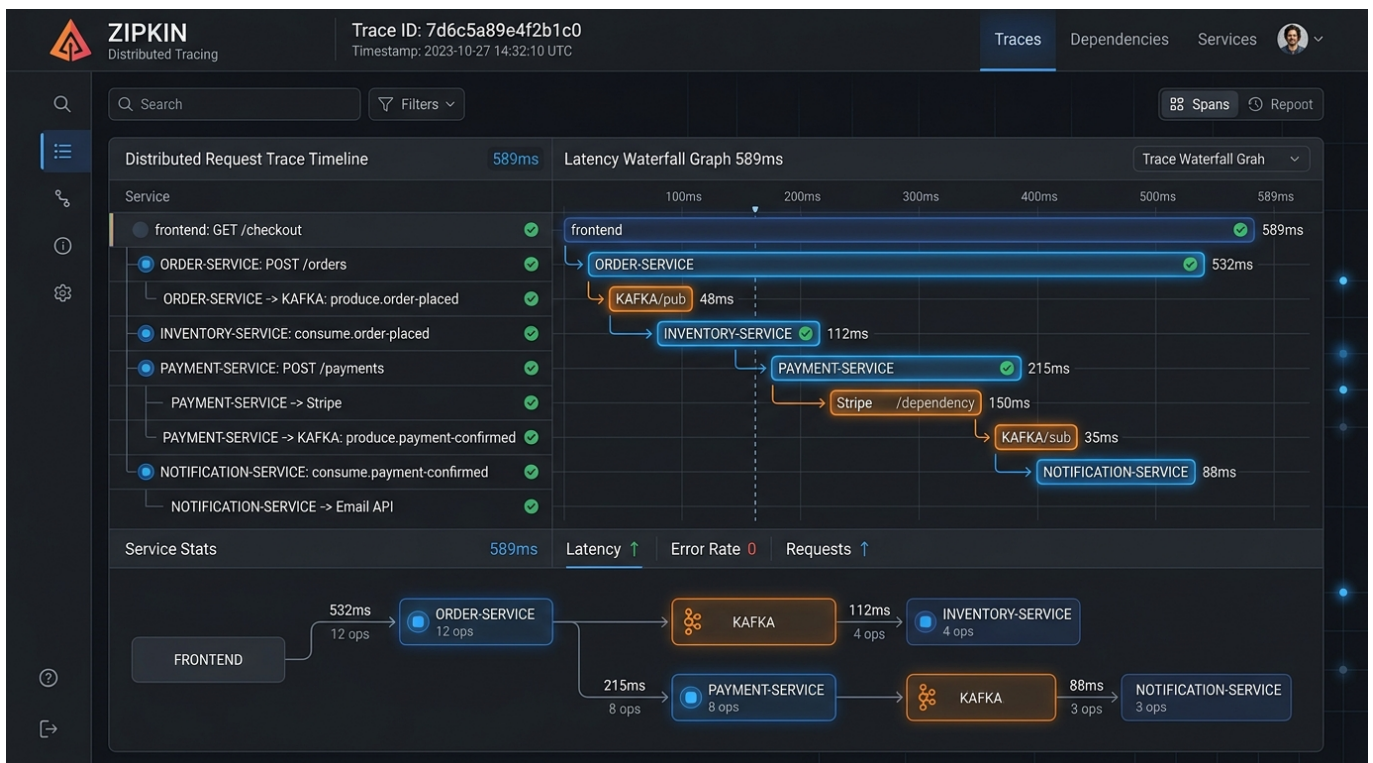


Figure 3: Zipkin Distributed Tracing Timeline Waterfall (<http://localhost:9411>)

4. Verified Transaction Runtime Output

```
GET http://localhost:8085/api/v1/orders/ORD-c04c1654
{
```

"id": 24, "orderNumber": "ORD-c04c1654", "customerId": "cust_new_1",

"productId": "prod_laptop_1", "quantity": 1, "totalAmount": 499.99,

"status": "CONFIRMED",

"inventoryReserved": true,

"paymentProcessed": true,

"createdAt": "2026-09-30T22:53:01Z"

}

5. Official Submission Sign-off

Author / Lead Developer: KANNEKANTI TEJA KIRAN

Submitted To: ZAALIMA DEVELOPMENT PVT LTD

Repository Link: <https://github.com/tejaa27/E-Commerce-microservies>

Local Project System Folder: C:\E-Commerce